

Оглавление

1	Особенности SQL.....	2
1.1	Дублирование кортежей.....	2
1.2	Монотонные запросы и операция деления.....	3
1.3	Отдельные особенности SQL.....	6
2	Трехзначная логика и null - значения.....	8
2.1	Отсутствующие значения и понятие null – значения в СУБД.....	8
2.2	Основы трехзначной логики	8
2.3	Операции с NULL - значениями.....	10
2.4	Особенности запросов с null - значениями	11
2.5	Советы по проектированию с учетом NULL-значений	14
3	Сложные запросы и их реализация	15
3.1	Построение подзапросов	15
3.2	Псевдо-таблицы	16
4	Статический и динамический SQL.....	18
4.1	Статический SQL и выполнение запросов	18
4.1.1	Выполнение SQL операций.....	18
4.1.2	Курсоры.....	19
4.2	Динамический SQL.....	20
4.2.1	Понятие и применение динамического SQL.....	20
4.2.2	Создание операторов	20
4.2.3	SQL – дескрипторы и динамические курсоры	22

1 ОСОБЕННОСТИ SQL

1.1 Дублирование кортежей

В реляционном отношении повторяющихся кортежей не может быть по определению, однако реляционные СУБД допускают такую ситуацию (кроме, как правило, базовых отношений). Рассмотрим пример 1.20.(a) с простым запросом, в результате которого появляются повторяющиеся кортежи.

Пример 1.20. (a)

```
select Doljnost as 'Должность'  
from TEmployers;
```

В результате выполнения запроса может быть получен следующий набор данных:

Должность

Специалист

Инженер

Главный инженер

Специалист

Ведущий инженер

Инженер

Очевидно, что в результате есть две пары повторяющихся кортежей: «Специалист» и «Инженер». Для обеспечения корректности результата необходимо использовать в запросе ключевое слово **DISTINCT**, исключающее повторения. Исключение повторяющихся кортежей проиллюстрировано примером 1.20.(b) и результатом его выполнения.

Пример 1.20.(b)

```
select distinct Doljnost as 'Должность'  
from TEmployers;
```

Результат представлен на тех же данных, что и примера 1.20(a):

Должность

Специалист

Инженер

Главный инженер

Ведущий инженер

1.2 Монотонные запросы и операция деления

Conjunctive Queries, монотонность запросов и операция деления

Запросы к реляционной схеме, состоящей из комбинаций выборки, проекции и соединения, называется CQ-запросы (Conjunctive Queries).

Это монотонные запросы.

```
SELECT Fname, Lname
FROM (EMPLOYEE JOIN WORKS_ON ON Ssn=Essn)
      JOIN PROJECT ON Pno=Pnumber
WHERE Dno=5
      AND Hours>=10
      AND Pname='ProductX'
```

Π - project
 $\bowtie$ - join
 σ - select

Запрос является *монотонным* тогда и только тогда, когда для двух БД $D_1 \subseteq D_2$ верно, что запрос $Q(D_1) \subseteq Q(D_2)$.

На практике используются в том числе немонотонные запросы.

Немонотонными являются операции **вычитания и деления** и для их реализации часто используются вложенные подзапросы.

(1) Выведите имена всех работников, которые работают над каждым проектом.

Почему эта задача не имеет решения в классе CQ мы можем понять, определив ключевые свойства самого запроса и класса CQ. CQ — класс моноotonно возрастающих запросов.

Задача (1) не является монотонной (*докажите это*).

Решение:

Если контр-примера к утверждению не существуют, то это утверждение всегда

верно. Формально: $\forall x : p(x) \equiv \neg \exists x : \neg p(x)$ — не существует x , такой что $p(x)$ неверен.

В задаче мы видим в явном виде квантор «для всех», и мы можем его эмулировать, используя двойное отрицание, то есть перефразируем запрос следующим образом: *вывести имена всех сотрудников, для которых не существуют проекта, над которым они бы не работали*

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS
  (SELECT *
   FROM PROJECT
   WHERE NOT EXISTS
     (SELECT *
      FROM WORKS_ON
      WHERE Pno = Pnumber
            AND Essn = Ssn))
```

Not exist
Counter-example

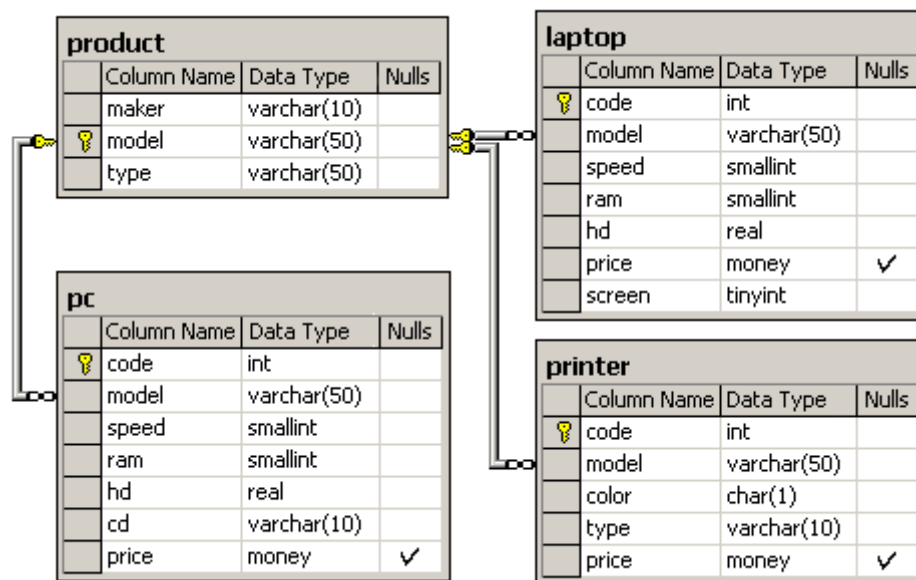
Этот же запрос выглядит невероятно просто, если бы у нас был $\forall$ (а он есть в реляционном исчислении):

$\{e.fname, e.lname \mid EMP(e) \text{ and } \forall p: PRJ(p) \Rightarrow \exists w: WORKS_ON(w) \text{ and } w.Pno = p.Pnumber \text{ and } e.Ssn = w.Essn\}$

```
Select Fname From Employer Where NotExist (
    Select * From Project Where NotExist (
        Select * From Workers_on Where Pno=Pnumber and Essn=SSn
    );
);
```

Рассмотрим пример других реализаций реляционного деления.

Рассмотрим следующую задачу.



Определить производителей, которые выпускают модели всех типов

Ключевым словом здесь является "всех", т.е. производитель в таблице Product должен иметь модели каждого типа, т.е. и PC, и Laptop, и Printer. Как раз для решения подобных задач в реляционную алгебру Коддом была введена специальная операция реляционного деления (DIVIDE BY).

С помощью этой операции наша задача решается очень просто:

```
1. Product [maker, type] DIVIDE BY Product [type]
```

Здесь квадратными скобками обозначается операция взятия проекции на соответствующие атрибуты.

Операция реляционного деления избыточна, т.е. она может быть выражена через другие операции реляционной алгебры. Возможно, поэтому ее нет в языке SQL. На примере решения сформулированной задачи посмотрим несколько приемов реализации операции реляционного деления на языке SQL.

Группировка

Если использовать тот факт, что, согласно описанию предметной области, типов продукции всего три, то мы можем выполнить группировку по производителю и подсчитать количество уникальных типов. Затем мы отберем только тех производителей, у которых это число равно трем.

```
1. SELECT maker
2. FROM Product
3. GROUP BY maker
4. HAVING COUNT(DISTINCT type) = 3;
```

Однако если число типов продукции произвольно, то такое решение будет правильным только при нынешнем состоянии базы данных, а не при любом возможном. А это значит, что мы должны константу заменить "переменной", значением которой будет текущее число типов, т.е. подзапросом:

```
1. SELECT maker
2. FROM Product
3. GROUP BY maker
4. HAVING COUNT(DISTINCT type) =
5.     (SELECT COUNT(DISTINCT type) FROM Product);
```

Разность

Если взять операцию разности BCEX имеющихся типов моделей и типов у конкретного производителя, то результирующая выборка не должна содержать строк.

```
1. SELECT DISTINCT maker
2. FROM Product Pr1
3. WHERE 0 = (SELECT COUNT(*) FROM
4. (SELECT type FROM Product
5. EXCEPT
6. SELECT type FROM Product Pr2
7. WHERE Pr2.maker = Pr1.maker
8. ) X );
```

Этот запрос можно написать короче, если воспользоваться тем свойством, что истинностное значение предиката ALL есть TRUE, если подзапрос не возвращает строк:

```
1. SELECT DISTINCT maker
2. FROM Product Pr1
3. WHERE type = ALL
4. (SELECT type FROM Product
5. EXCEPT
6. SELECT type FROM Product Pr2
7. WHERE Pr2.maker = Pr1.maker
8. );
```

Для искомых производителей список типов в предикате ALL будет пуст (предикат равен TRUE). В остальных случаях он будет содержать типы моделей, отсутствующие у производителя из внешнего запроса, поэтому операция сравнения (равенство "=") для всех его моделей даст FALSE.

Существование

Не должно существовать такого типа продукции, которого бы не было у искомого производителя. Это основной логический подход, пример которого был приведен выше.

```
1. SELECT DISTINCT maker
2. FROM Product Pr1
3. WHERE NOT EXISTS
4.     (SELECT type
5.       FROM Product
6.       WHERE type NOT IN
7.           (SELECT type
8.             FROM Product Pr2
9.             WHERE Pr1.maker = Pr2.maker
10.          )
11.    );
```

Кроме первого варианта с группировкой все остальные решения используют *коррелирующие подзапросы* для определения множества типов моделей производителя из основного запроса.

Следует также отметить, что решение с группировкой не подойдет для случая, когда требуется выполнить деление не на все множество имеющихся типов, а на некоторое их подмножество. Например, если требуется найти производителей, у которых множество типов включает в себя (или совпадает) множество типов, определяемое некоторыми критериями. Другие же приемы можно адаптировать для решения подобной задачи.

1.3 Отдельные особенности SQL

Двойные и одинарные кавычки в части SELECT

Двойные кавычки могут использоваться в языке запросов, более того, результат применения двойных и одинарных кавычек может быть совсем разным.

Двойные кавычки	Одинарные кавычки
SELECT name "Имя товара" FROM goods	SELECT name 'Это данные' FROM goods WHERE id = 1
Имя товара Тапочки Подушка Одеяло Наволочка Простынка	name Это данные

Сравнение наборов данных

Быть может вы знаете, а может нет, что в SQL, при построении запросов, можно проводить сравнение наборов данных. Пример:

```
SELECT *
```

```
FROM goods
```

```
WHERE (name, price, availability) = ('Наволочка', 400.00, FALSE)
```

-- или его аналог

```
SELECT *
```

```
FROM goods
```

```
WHERE name = 'Наволочка' AND price = 400.00 AND availability = FALSE
```

Результат выполнения этих запросов будет одинаков, например:

id	name	description	price	articul	act_time	availability
4	Наволочка	Серая	400.00	AR-75	2017-01-04 04:04:04.04	false

То есть, *сравнение блоков, данных аналогично поэлементному сравнению значений, входящих в эти блоки*. Причем в блоке могут фигурировать как константы, так и значения атрибутов или переменных.

Операторы ANY, SOME, ALL

Рассмотрим еще применение таких нечастых SQL – операторов, как операторы ANY, SOME и ALL. Не смотря на то, что операторов три, типа поведения СУБД – сервера при встрече с ними всего два. Рассмотрим их.

Стратегия 1	Стратегия 2																
ALL	ANY, SOME																
SELECT id, name FROM goods WHERE id > ALL (SELECT id FROM goods WHERE availability)	SELECT id, name FROM goods WHERE id > ANY (SELECT id FROM goods WHERE availability)																
<table><tr><td>id</td><td>name</td></tr><tr><td>4</td><td>Наволочка</td></tr><tr><td>5</td><td>Простынка</td></tr></table>	id	name	4	Наволочка	5	Простынка	<table><tr><td>id</td><td>name</td></tr><tr><td>2</td><td>Подушка</td></tr><tr><td>3</td><td>Одеяло</td></tr><tr><td>4</td><td>Наволочка</td></tr><tr><td>5</td><td>Простынка</td></tr></table>	id	name	2	Подушка	3	Одеяло	4	Наволочка	5	Простынка
id	name																
4	Наволочка																
5	Простынка																
id	name																
2	Подушка																
3	Одеяло																
4	Наволочка																
5	Простынка																
Смысловой аналог: AND	Смысловой аналог: OR																

2 ТРЕХЗНАЧНАЯ ЛОГИКА И NULL - ЗНАЧЕНИЯ

2.1 Отсутствующие значения и понятие null – значения в СУБД

Принято считать, что NULL-значения представляют пока что неизвестные значения, которые могут быть позднее заменены реальными значениями. Это единственный способ обозначить отсутствие значения.

1. Отсутствующее (не заданное) значение.
2. это Not Applicable (в формах и электронных таблицах он обозначается N/A). *Например, атрибут цвета волос лысого человека имеет значение NULL. Последнее отражает отсутствие значений и взято из домена цвета волос. В то же время цвет перьев этого человека также равен NULL, но это уже означает тип Not Applicable (неприемлемо).*
3. Отсутствует не только значение атрибута, но и сам атрибут.
(Пример, когда для проведения статистического анализа группируются все медицинские записи. В таблице с результатами вы не найдете ни одного беременного мужчину. Таким образом, при обработке случаев беременности перед программистом возникает выбор.
 - a. Например, он может в итоговой таблице создать столбец "number of pregnancies" (число беременностей), для строк, в которых sex="male", в этом столбце ставить значение 0 или NULL, а затем следить за выполнением этого правила целостности с помощью конструкции CHECK().
 - b. Вторая возможность — создать столбец "medical condition", а рядом с ним еще один "number of occurrences" (количество экземпляров). Другая конструкция CHECK() будет следить за отсутствием мужских беременностей.

2.2 Основы трехзначной логики

В SQL предусмотрена трехзначная логика — TRUE, FALSE и UNKNOWN. Последнее значение возникает в результате использования в сравнениях и других предикатах NULL-значений.

Имейте в виду, что это логическое значение, **оно не совпадает с NULL, которое представляет собой значение данных**. Именно поэтому на языке SQL пишут (x IS [NOT] NULL), а не (x = NULL).

Таблица. Трехзначная логика SQL

A	B	Not A	A and B	A or B
TRUE	TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE	TRUE
TRUE	Null	FALSE	Null	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE
FALSE	FALSE	TRUE	FALSE	FALSE
FALSE	Null	TRUE	FALSE	Null
Null	TRUE	Null	Null	TRUE
Null	FALSE	Null	FALSE	Null
Null	Null	Null	Null	Null

Наличие неопределенных значений в данных, если их не учитывать, может приводить к неожиданному для разработчика результатам выполнения простых на вид запросов.

Произведем подсчет суммы окладов всех сотрудников при выполнении запроса соединения (пример 1.21.(a)). При этом в сумму будут добавлены сведения из всех кортежей, включая тех, где фамилия работника Null – то есть, работник отсутствует и, не смотря на наличие такой должности, зарплата никому не выплачивается.

Пример 1.21. (a)

```
select sum ( TDoljnost.Oklad) as N'Сумма окладов'  
from TDoljnost full join TEmployers  
on TEmployers.Doljnost=TDoljnost.Doljnost;
```

Результат запроса:

```
Сумма окладов  
-----  
315000,00
```

Пример 1.21. (b)

```
select sum ( TDoljnost.Oklad)  
as N'Сумма окладов (учет NULL)'  
from TDoljnost full join TEmployers  
on TEmployers.Doljnost=TDoljnost.Doljnost  
where TEmployers.Employer is not null;
```

Результат запроса:

```
Сумма окладов (учет NULL)  
-----  
249000,00
```

В примере 1.21.(b) учтены отсутствующие сотрудники при помощи части WHERE и результат суммирования окладов будет уже другим.

Также при обработке данных для корректного отображения пользователю Null – значения могут быть заменены. Пример скрипта, заменяющего неопределенные данные в нашем примере представлен в примере 1.22.

Пример 1.22.

```
select COALESCE(TEmployers.Employer,  
N'На должности нет работника') as N'Работник',  
COALESCE(TDoljnost.Doljnost,  
N'Должность не указана') as N'Должность',  
COALESCE(TDoljnost.Oklad, 0) as N'Оклад'  
from TDoljnost full join  
TEmployers on TEmployers.Doljnost=TDoljnost.Doljnost
```

Тогда результат выполнения операции будет таким, как представлен на рисунке – нижний результат (сравните с верхним результатом на том же рисунке без замены данных).

Employer	Doljnost	Oklad
NULL	NULL	35000,00
Семенов	Ведущий инженер	50000,00
Карпов	Главный инженер	55000,00
Иванова	Инженер	42000,00
Стеганов	Инженер	42000,00
NULL	NULL	31000,00
Иванов	Специалист	30000,00
Каримова	Специалист	30000,00
Кречетов	NULL	NULL



Работник	Должность	Оклад
На должности нет работника	Администратор	35000,00
Семенов	Ведущий инженер	50000,00
Карпов	Главный инженер	55000,00
Иванова	Инженер	42000,00
Стеганов	Инженер	42000,00
На должности нет работника	Менеджер	31000,00
Иванов	Специалист	30000,00
Каримова	Специалист	30000,00
Кречетов	Должность не указана	0,00

Рисунок. Замена неопределенных значений

Учитывать неопределенные и повторяющиеся значения необходимо для корректного получения данных в результате SQL – запросов.

2.3 Операции с NULL - значениями

Сравнение NULL-значений

Работать с NULL-значениями кажется неестественным. Их нельзя сравнивать друг с другом с помощью операторов сравнения (равно, неравно, меньше, больше и т.д.), так что вместо двузначной логики мы получаем трехзначную. Это непривычно для большинства программистов. Если выполнить оператор:

```
SELECT * FROM SomeTable WHERE SomeColumn = 2;
```

а затем:

```
SELECT * FROM SomeTable WHERE SomeColumn <> 2;
```

то можно получить все строки таблицы SomeTable. Однако для достижения такого результата следует выполнить еще и оператор:

```
SELECT * FROM SomeTable WHERE SomeColumn IS NULL;
```

Предикат IS [NOT] NULL возвращает только значения TRUE или FALSE.

Математические выражения и NULL-значения

Если NULL встречается в арифметических выражениях (+, -, *, /), то они дают в ответе также NULL. Подробнее ниже.

Функции и NULL-значения

Большинство разработчиков одинаково обрабатывают NULL-значения в функциях, предлагаемых ими как расширения стандарта SQL.

Например, косинус от NULL также будет NULL.

1. NULLIF(V1, V2) возвращает NULL, если первый параметр равен второму. Функция эквивалентна следующей спецификации CASE:

```
CASE WHEN (V1 = V2) THEN NULL ELSE V1 END
```

2. COALESCE(V1, V2, V3, ..., Vn) просматривает список параметров слева направо и возвращает первый из них, не равный NULL. Если все значения равны NULL, функция возвращает также NULL.

2.4 Особенности запросов с null - значениями

Многие забывают о том, что подзапрос часто содержит сравнение с NULL. Взгляните на следующие две таблицы:

```
CREATE TABLE Table1 (col1 INTEGER);  
INSERT Table1 (col1) VALUES (1);  
INSERT Table1 (col1) VALUES (2);  
CREATE TABLE Table2 (col1 INTEGER);  
INSERT Table2 (col1) VALUES (1);  
INSERT Table2 (col1) VALUES (2);  
INSERT Table2 (col1) VALUES (3);  
INSERT Table2 (col1) VALUES (4);  
INSERT Table2 (col1) VALUES (5);
```

Обратите внимание, что их столбцы допускают NULL. Выполните такой запрос:

```
SELECT col1  
FROM Table2  
WHERE col1 NOT IN (SELECT col1 FROM Table1);
```

Result

col1

3

4

5

Введите теперь NULL и снова выполните его:

```
INSERT Table1 (col1) VALUES (NULL);  
SELECT col1  
FROM Table2  
WHERE col1 NOT IN (SELECT col1 FROM Table1);
```

Результат будет пустым. Это не совсем понятно, но правильно.

Предикат NOT IN определяется следующим образом:

```
SELECT col1
FROM Table2
WHERE NOT (col1 IN (SELECT col1 FROM Table2),
```

Предикат IN определен так:

```
SELECT col1
FROM Table2
WHERE NOT (col1 = ANY (SELECT col1 FROM Table1));
```

Это означает:

```
SELECT col1
FROM Table2
WHERE NOT ((col1 = 1)
           OR (col1 = 2)
           OR (col1 = 3)
           OR (col1 = 4)
           OR (col1 = 5)
           OR (col1 = NULL));
```

Последнее выражение всегда равно UNKNOWN, поэтому с учетом законов Де Моргана запрос фактически выглядит следующим образом:

```
SELECT col1
FROM Table2
WHERE ((col1 <> 1)
      AND (col1 <> 2)
      AND (col1 <> 3)
      AND (col1 <> 4)
      AND (col1 <> 5)
      AND UNKNOWN);
```

Взглянув на приведенные выше таблицы значений логических операторов, вы поймете, что выражение всегда будет равно UNKNOWN, которое отвергается в условии поиска в конструкции WHERE.

Обобщающая таблица:

Область	Пример	Результат
Сравнение с NULL	select case when null is null then 'YES' else 'NO' end from dual; -- YES	IS NULL вернёт истину, если операнд имеет значение NULL и ложь, если он им не является
	select case when 'a' is NOT null then 'YES' else 'NO' end from dual; -- YES	IS NOT NULL действует наоборот
	DECODE (<Контрольное значение>, <Значение1>, <Значение2>, ...<Дефолтное значение>) select decode(null , 1, 'ONE', null, 'EMPTY', 'DEFAULT') - это условие будет истинным	Oracle: считает два NULLа эквивалентными друг другу

Логические операции и NULL	select 1 from dual where dummy = null *** таблица истинности	запрос не вернёт записей (dummy = null - ЛОЖЬ)
Операторы IN и NOT IN	create table t as select column_value a from table(sys.odcnumberlist(1,2,3,null));	Для тестов создадим таблицу T с одним числовым столбцом A и четырьмя строками: 1, 2, 3 и NULL
	select * from t where a in(1,2,null); -- вернёт [1,2]	вычисление предиката "A"=TO_NUMBER(NULL) вернуло состояние НЕИЗВЕСТНО
	select * from t where a in(1,2) or a is null; -- вернёт [1,2,NULL]	
	select * from t where a not in(1,2,null); -- no rows selected	Из-за особенностей трёхзначной логики NOT IN вообще не дружит с NULLами: как только NULL попал в условия отбора, данных не ждите ¹
NULL и пустая строка	Не равны, но! Oracle отходит от стандарта ANSI SQL и провозглашает эквивалентность NULLа и пустой строки	
Математика NULLa	null + 10, null * 10, abs(null), sign(null) ...	-- неизвестно
	select null 'AA' null 'BB' null from dual; -- AABBB	вы можете добавить NULL к строке и это её не изменит
Агрегатные функции	create table agg(id int, n int); insert into agg values(1, 1); insert into agg values(2, 3); insert into agg values(3, null); insert into agg values(4, 2); commit;	Почти все агрегатные функции, за исключением COUNT (и то не всегда), игнорируют пустые значения при расчётах
	select sum(n) from agg; -- 6	Пустые значения игнорируются агрегатами
	select count(*) from agg; -- 4 select count(1+1) from agg; -- 4 select count(user) from agg; -- 4 С выражением: select count(n) from agg; -- 3 select count(id) from agg; -- 4 select count(abs(n)) from agg; -- 3	если используется в виде COUNT (*) или COUNT(константа), будет учитывать пустые значения. Однако, если она используется в виде COUNT(выражение), то пустые значения будут игнорироваться
	select avg(n) from agg; -- (1 + 3 + 2) / 3 = 2 select avg(nvl(n,0)) from agg; -- (1 + 3 + 0 + 2) / 4 = 1.5	Для решения подобных проблем есть стандартное решение — воспользоваться функцией NVL
Агрегатные функции и пустые (NULL)	Набор данных только из NULLов: select sum(n) from agg where n is null; select avg(n) from agg where n is null;	Агрегатные функции возвращают состояние НЕИЗВЕСТНО, если они применяются к набору только из NULLов. -- НЕИЗВЕСТНО

¹ Постройте дерево выражения и определите, почему результат будет NULL

наборы данных	<code>select count(n) from agg where n is null;</code>	<code>-- 0</code>
	Пустой набор данных: <code>select sum(n) from agg where 1 = 0;</code> <code>select avg(n) from agg where 1 = 0;</code>	Агрегатные функции возвращают состояние НЕИЗВЕСТНО, если они применяются к пустому набору данных. <code>-- НЕИЗВЕСТНО</code>
	<code>select count(n) from agg where 1 = 0;</code>	<code>-- 0</code>

2.5 Советы по проектированию с учетом NULL-значений

1. При каждой возможности рекомендуется определять все ваши базовые таблицы с ограничениями NOT NULL во всех столбцах. NULL-значения могут смутить тех, кто не знаком с SQL, кроме того, для их реализации применяется дополнительный бит где-нибудь в строке, а не в столбце. Это повышает расход дискового пространства, а также снижает производительность индексирования и поиска.

2. **Значения NULL недопустимы в столбцах первичного ключа.** Подумайте о том, что мог бы означать первичный ключ, частично равный NULL. Использование NULL-значений в SQL всегда предполагало, что они когда-либо в будущем могут быть заменены реальными значениями. NULL в качестве ключа означал бы, что СУБД не может определить, дублирует ли первичный ключ значение, уже имеющееся в таблице.

3. NULL-значений следует избегать и во внешних ключах. SQL допускает подобные связи, но они могут привести к потере информации в запросах с соединениями. Например, если вы работаете с кодами номеров деталей в реестре и таблица *Orders* ссылается на эти коды как на внешний ключ, вы не сможете использовать список деталей, содержащий значения NULL.

4. **Значения NULL недопустимы в полных системах кодирования.** Например, все сотрудники — это люди, а люди делятся исключительно на мужчин и женщин.

5. Значения по умолчанию по возможности не должны быть NULL.

а. Если вы знаете значения по умолчанию для количественных параметров, с помощью конструкции DEFAULT их можно автоматически вводить в столбец, не меняя структуры таблицы. Обычно в таких случаях для числовых столбцов используется ноль, а для символьных — пробелы, но вы можете взять любое другое допустимое значение.

б. Показать, что некоторые значения отсутствуют, можно соответственно спроектировав системы кодирования. Например, в соответствии со стандартом ISO код пола равен 0 для неизвестного пола, 1 для мужчины, 2 для женщины и 9 в случае неприемлемости (например, он неприменим к корпорациям). Такие обозначения распространены в числовых системах кодирования.

с. При работе с именами для обозначения отсутствующих значений лучше использовать специальную фиктивную строку, а не NULL. Это позволяет, например, для различения имен составить список типа 'John Doe #1', 'John Doe #2' и т.д.; в случае NULL это невозможно.

Иногда в количественных параметрах NULL применять необходимо.

Между нулем и неизвестной величиной есть разница, аналогичная различию между пустым бензобаком и отсутствием автомобиля вообще. Отрицательные числа также не подходят для обозначения отсутствующих значений — это слишком усложняет расчеты.

Однако может быть необходимо использовать NULL в полях дат, если неудобно вводить дату по умолчанию. Например:

- не имеет смысла вводить дату по умолчанию в поле дня рождения;
- если гарантия не имеет срока давности, NULL можно считать символом вечности.

3 СЛОЖНЫЕ ЗАПРОСЫ И ИХ РЕАЛИЗАЦИЯ

3.1 Построение подзапросов

Условно подзапросы иногда подразделяют на **три типа**, каждый из которых является сужением предыдущего:

1. табличный подзапрос, возвращающий набор строк и столбцов;
2. подзапрос строки, возвращающий только одну строку, но, возможно, несколько столбцов (такие подзапросы часто используются во встроенном SQL);
3. скалярный подзапрос, возвращающий значение одного столбца в одной строке.

Подзапрос позволяет решать следующие задачи:

1. определять набор строк, добавляемый в таблицу на одно выполнение оператора INSERT ;
2. определять данные, включаемые в представление, создаваемое оператором CREATE VIEW ;
3. определять значения, модифицируемые оператором UPDATE ;
4. указывать одно или несколько значений во фразах WHERE и HAVING оператора SELECT ;
5. определять во фразе FROM таблицу как результат выполнения подзапроса ;
6. применять *коррелированные подзапросы*. Подзапрос называется **коррелированным**, если запрос, содержащийся в предикате, имеет ссылку на значение из таблицы (внешней к данному запросу), которая проверяется посредством данного предиката.

Некоторые СУБД (например, СУБД Oracle) позволяют на основе подзапроса создавать новые таблицы с помощью оператора CREATE TABLE.

Подзапрос	Результат	Варианты действия
SELECT * from tbl1 WHERE f2= (SELECT f2 FROM tbl2 WHERE f1=1);	Одна строка	Ошибка или корректное сравнение в зависимости от СУБД
	Набор (несколько) строк	Ошибка синтаксиса или выполнения
	Пустое множество строк	UNKNOWN
SELECT f1,f2,f3 FROM tbl1 WHERE f2> // f2 IN (SELECT AVG(f2) FROM tbl1);	Одна строка	Ошибка без IN
	Набор (несколько) строк	TRUE, если хотя бы одно из значений, возвращаемых подзапросом, удовлетворяет условию
	Пустое множество строк	

Коррелированные подзапросы

В операторе SELECT из внутреннего подзапроса можно сослаться на столбцы внешнего запроса, указанного во фразе SELECT. Такой подзапрос выполняется для каждой строки таблицы, определяя условие ее вхождения в формируемый результирующий набор.

Пример	Описание
SELECT * from tbl1 t1 WHERE f2 IN (SELECT f2 FROM tbl2 t2 WHERE t1.f3=t2.f3);	для каждой строки таблицы tbl1 будет проверяться условие, что значение поля f2 совпадает со значением строки таблицы tbl2, где значение поля f3 равно значению поля f3 внешней таблицы (tbl1)
SELECT * from tbl1 t_out WHERE f2< (SELECT AVG(f2) FROM tbl1 t_in WHERE t_out.f1= t_in.f1);	Использование данных внешней таблицы и алиасов (ссылок) для нескольких обращений к одной таблице

Пример	Описание
<pre>SELECT f1, COUNT(*), SUM(f2) from tbl1 t1 GROUP BY f1 HAVING SUM(f2)> (SELECT MIN(f2)*4 FROM tbl1 t1_in WHERE t1.f1=t1_in.f1);</pre>	во фразе HAVING можно использовать только агрегирующие функции, (каждый раз на момент выполнения подзапроса в качестве проверяемой строки, к значениям которой имеет доступ подзапрос, выступает результат группирования строк на основе агрегирующих функций основного запроса)

Для того чтобы проверить, существуют ли строки, удовлетворяющие конкретному условию подзапроса, применяется оператор EXISTS.

Пример	Описание
<pre>SELECT f1,f2,f3 from tbl1 WHERE EXISTS (SELECT * FROM tbl1 WHERE f4='10/11/2003');</pre>	запрос будет формировать не пустой результирующий набор только в том случае, если в какое-либо значение столбца f4 таблицы была занесена дата '10/11/2003'

Преимущество применения оператора EXISTS с результатами подзапроса состоит в том, что *подзапрос может возвращать как множество строк, так и множество столбцов*.

При коррелированном подзапросе оператор EXISTS будет **вычисляться каждый раз** для каждой строки внешнего запроса.

3.2 Псевдо-таблицы

Эта разработка появилась в стандарте SQL – 92. Рассмотрим пример:

```
SELECT      mock.nickname "Прозвище",
            (CASE WHEN mock.huff THEN 'Да' ELSE 'Нет' END) "Есть?"
FROM        (SELECT name AS nickname, availability AS huff FROM goods) mock
```

Обратите внимание, что в разных диалектах языка запросов последнее предложение может выглядеть так:

```
FROM        (SELECT name AS nickname, availability AS huff FROM goods) as mock
```

А само название псевдо-таблицы **mock** быть (как в T-SQL) или не быть (PL/SQL) обязательным.

Результат приведенного запроса имеет вид:

Прозвище	Есть?
Тапочки	Да
Подушка	Да
Одеяло	Да
Наволочка	Нет
Простынка	Нет

При таком построении запроса *отношение mock является виртуальным*. Оно не создается на физическом диске, а размещается (при возможности) в оперативной памяти и используется как промежуточное при реализации запроса.

Использовать виртуальные запросы можно и это делается достаточно широко, однако, стоит помнить, что **оптимизируются они хуже, чем обычные** – не все СУБД могут преобразовать такие синтаксические конструкции в операции над данными наиболее оптимальным образом.

Однако, в отличие от *промежуточных таблиц*, определенных как *переменные до запроса*, такие решения оптимизируются лучше и в целом быстрее при использовании промежуточной таблицы «один раз».

Создание псевдо - таблицы «на лету»

Язык запросов позволяет создать таблицу из данных «на лету». Как правило, это нужно при тестировании баз данных – когда соответствующих таблиц нет, создавать их достаточно хлопотно, но нужно, например, протестировать запрос и достаточно короткого блока данных.

Пример:

```
SELECT name "Имя товара", price "Цена"  
FROM (VALUES ('Тапочки', 100.00), ('Подушка', 200.00)) AS goods(name, price)
```

Имя товара	Цена
Тапочки	100.00
Подушка	200.00

От предыдущего примера этот отличается тем, что часть FROM использует не существующие таблицы, а наборы значений для создания псевдо-отношения.

4 СТАТИЧЕСКИЙ И ДИНАМИЧЕСКИЙ SQL

4.1 Статический SQL и выполнение запросов

4.1.1 Выполнение SQL операций

Фактически, чтобы приложение могло обращаться к базе данных, существуют четыре основных варианта:

1. встраивание в код некоторого языка программирования SQL-операторов (**статический SQL**); Встраиваемый SQL-оператор указывается после фразы EXEC SQL.
2. формирование в процессе выполнения программы на некотором языке программирования кода SQL-операторов и дальнейшего их выполнения (**динамический SQL**);
3. вызов из программ, написанных на других языках программирования, SQL-модулей;
4. использование API (Application Programming Interface), позволяющего реализовывать работу с базой данных через предоставляемый набор функций.

API может быть целевым, предоставленным производителем коммерческой СУБД для работы именно с этой базой данных, или межплатформенным, реализующим унифицированные средства доступа к СУБД различных производителей. К такому API относятся ODBC (Open DataBase Connectivity) и SQL/CLI (SQL Call Level Interface).



Статический SQL встраивается в виде препроцессорной обработки в традиционные языки программирования. Операторы SQL обрабатываются прекомпилятором. В SQL-операторах могут использоваться переменные из прикладной программы.

СУБД возвращает информацию об ошибках через специальные переменные².

На следующей схеме изображен процесс выполнения программы, содержащей операторы встроенного SQL.

² структуру SQL Communication AREA (SQLCA), переменную SQLSTATE или SQLCODE

Построение плана выполнения на этапе компиляции выполнимого кода отличает *статический встроенный SQL* от *динамического SQL*.

При выполнении программы со встроенным SQL могут быть использованы привилегии как выполняющего, так и владеющего пользователя (того, кто создал выполнимый файл). В современных коммерческих СУБД, как правило, относительно хранимых в базе данных модулей, существует возможность назначения привилегии EXECUTE таким образом, чтобы при выполнении модуля использовались привилегии его владельца.

4.1.2 Курсоры

Под **курсором**, как правило, понимают *получаемый при выполнении запроса результирующий набор и связанный с ним указатель текущей записи*. Курсор - это объект, связанный с определенной областью памяти.

Существуют явные и неявные курсоры.

- **Явный курсор** имеет **имя** и перед использованием должен быть **объявлен**.
- Неявный курсор создается автоматически и его нельзя повторно открыть без перекompиляции оператора запроса.

Объявление курсора выполняется оператором DECLARE CURSOR, в котором фраза FOR определяет запрос, ассоциируемый с данным курсором.

Пример	Описание
EXEC SQL DECLARE c1 CURSOR FOR SELECT f1, f2, f3 FROM tbl1 WHERE f2>100;	создает курсор c1 на базе таблицы tbl1. При объявлении курсора выполнения запроса не происходит.
EXEC SQL OPEN CURSOR c1;	инициирует курсор, выполнив определенный в нем оператор SELECT

Выполнение запроса и создание курсора инициируется оператором OPEN CURSOR

Приложение получает доступ к данным курсора при последовательном извлечении строк результирующего набора в переменные приложения.

Для освобождения памяти, выделенной под курсор, его следует закрыть, выполнив оператор CLOSE CURSOR.

Для работы с NULL-значениями в курсорах предусмотрены индикаторные переменные, которые могут использоваться для:

- определения извлекаемого NULL-значения;
- внесения NULL-значения в таблицу;
- фиксирования усекаемых строк.

Если в результате выполнения оператора FETCH или оператора SELECT (*возвращающего одну строку*) извлекаемые данные **принимают значение NULL**, то,

- считается, что SQL-оператор выполнен с ошибкой
- в переменную будет записано значение, отличное от NULL (зависит от типа переменной).

Для предотвращения таких ситуаций применяются *индикаторные переменные*, указываемые после INTO-переменной через символ двоеточия (или INDICATOR:).

Если индикаторная переменная принимает отрицательное значение, значит, столбец содержит значение NULL. По умолчанию до выполнения оператора индикаторной переменной присваивается значение 0.

4.2 Динамический SQL

4.2.1 Понятие и применение динамического SQL

Динамический SQL — это операторы SQL, которые создаются во время выполнения; например, приложение может разрешать пользователям вводить свои собственные запросы.

Динамический SQL — это метод программирования, который позволяет динамически создавать операторы SQL во время выполнения. Вы можете создавать более универсальные, гибкие приложения, используя динамический SQL, поскольку полный текст оператора SQL может быть неизвестен при компиляции.

Преимущество использования статического SQL заключается в *оптимизации оператора*, что обеспечивает высокую производительность приложения. Планы доступа для динамических операторов создаются во время выполнения, их необходимо подготовить в приложении, но это не единственные различия между ними.

Причина	Пример	Реакция
нужен запрос, в котором имя таблицы воспринимается как параметр (чтобы можно было его задавать)	<pre>DECLARE @TableName VarChar(200) --объявил переменную SET @TableName = "Products" --присвоил ей значение SELECT * FROM @TableName --выдаст ошибку</pre>	Синтаксически в поле FROM нельзя подставлять переменную. Чтобы избежать ошибки, надо этот запрос сделать динамическим.

Сравнение статического и динамического подходов:

Static (Embedded) SQL	Dynamic (Interactive) SQL
В статическом SQL способ доступа к базе данных предопределен во встроенном операторе SQL.	В динамическом SQL способ доступа к базе данных определяется во время выполнения.
Более быстро и эффективно.	Менее быстро и эффективно.
Операторы SQL компилируются во время компиляции.	Операторы SQL компилируются во время выполнения.
Разбор, Валидация, Оптимизация и Генерация плана приложения выполняются во время компиляции.	Разбор, Валидация, Оптимизация и Генерация плана приложения выполняются во время выполнения.
EXECUTE IMMEDIATE, EXECUTE и PREPARE не используются.	EXECUTE IMMEDIATE, EXECUTE и PREPARE используются.
Менее гибок.	Более гибок

4.2.2 Создание операторов

Операторы динамического SQL - в отличие от операторов встроенного SQL - формируются не на этапе компиляции, а на этапе выполнения приложения³.

Операторы динамического SQL формируются как текстовые переменные.

³ Динамический SQL может применяться совместно с ODBC API или в рамках SQL/CLI, представляющего собой расширенный уровень соответствия стандарта SQL-99.

Например:

```
Stmt1:='SELECT * FROM tbl1';
```

Операторы динамического SQL можно использовать:

- однократно, производя за один шаг компиляцию и выполнение оператора. Будем называть такое применение *одношаговым интерфейсом*;
- многократно, разделяя процесс компиляции оператора, на котором строится план выполнения, и процесс непосредственного выполнения оператора. Будем называть такое применение *многошаговым интерфейсом*.

Одношаговый интерфейс

одношаговый интерфейс реализуется SQL-оператором EXECUTE IMMEDIATE, который имеет в стандарте SQL-92 следующее формальное описание:

```
EXECUTE IMMEDIATE :variable;
```

На оператор, указываемый переменной (variable), накладываются следующие ограничения:

- оператор не может использовать INTO-переменные;
- оператор не может использовать переменные связи.

Следующий пример иллюстрирует применение динамического SQL с одношаговым интерфейсом:

```
stmt_str := 'INSERT INTO ' || table_name ||  
            ' values (:f1, :f2, :f3);'  
  
EXEC SQL EXECUTE IMMEDIATE :stmt_str;
```

Многошаговый интерфейс

Оператор EXECUTE IMMEDIATE удобен для одноразового выполнения, но при необходимости неоднократного выполнения, например в цикле одного и того же оператора, но с различными параметрами, более эффективно использовать многошаговый интерфейс, реализуемый операторами PREPARE и EXECUTE.

При выполнении оператора PREPARE, указываемый им SQL-оператор передается в СУБД. Далее выполняется синтаксический разбор оператора и строится план выполнения. После этого при каждом выполнении оператора EXECUTE на сервер передаются:

- уже "откомпилированный" SQL-оператор.
- значения переменных связи (если есть), (в частности, для вычисления *WHERE*).⁴

Оператор PREPARE имеет в стандарте SQL-92 следующее формальное описание:

```
PREPARE [ GLOBAL | LOCAL ] operator_sql FROM string_variable;
```

- operator_sql - определяет идентификатор SQL-оператора, указываемый далее для выполнения в операторе EXECUTE или для включения в курсор.
- string_variable указывает строку, содержащую динамически сформированный текст SQL-оператора.

Например:

```
stmt_str := 'INSERT INTO ' || table_name ||  
            ' values (:f1, :f2, :f3);'  
  
EXEC SQL PREPARE GLOBAL stmt1 FROM :stmt_str;
```

⁴ Динамические параметры можно использовать как во встроенном SQL, так и в динамическом SQL.

Фразы GLOBAL и LOCAL определяют область видимости оператора:

- GLOBAL указывает, что оператор с данным идентификатором доступен всем процессам данного сеанса работы с СУБД
- LOCAL ограничивает доступ рамками данного выполняемого модуля (значение по умолчанию).

Если создаются два одноименных оператора, но один как GLOBAL, а другой - как LOCAL, то СУБД создает *два отдельных плана выполнения* как для разных операторов. В противном случае при компиляции оператора с уже существующим именем просто строится *новый план* выполнения оператора.

- Для освобождения подготовленного SQL-оператора используется оператор DEALLOCATE PREPARE, который освобождает все ресурсы, занимаемые подготовленным SQL-оператором.
- Для выполнения откомпилированного SQL-оператора используется оператор EXECUTE

4.2.3 SQL – дескрипторы и динамические курсоры

SQL-дескрипторы

SQL-дескриптор - это область, которая временно создается СУБД для хранения информации о параметрах откомпилированного SQL-оператора. Для каждого откомпилированного SQL-оператора создается своя область SQL-дескриптора.

SQL-дескрипторы могут быть использованы для следующих целей:

- определения параметров при выполнении SQL-оператора;
- получения информации об INTO-переменных;
- применения в динамических курсорах.

Работа с SQL-дескриптором состоит из следующих этапов:

1. Создание SQL-дескриптора оператором ALLOCATE DESCRIPTOR ;
2. Задание значений для SQL-дескриптора может выполняться:
 - при выполнении оператора SET DESCRIPTOR ;
 - при выполнении оператора DESCRIBE для автоматической установки значений;
3. Использование значений динамических параметров:
 - извлечение значений параметров при выполнении оператора GET DESCRIPTOR ;
 - использование значений параметров при выполнении оператора EXECUTE ;
4. Освобождение SQL-дескриптора оператором DEALLOCATE DESCRIPTOR.

Динамические курсоры

В динамическом SQL можно использовать не только курсоры встроенного SQL (создаваемые статически оператором DECLARE CURSOR), но и два дополнительных типа курсоров:

- объявляемые курсоры (declared cursors), создаваемые как DECLARE CURSOR ;
- размещаемые курсоры (allocated cursors), создаваемые как ALLOCATE CURSOR . Этот тип курсоров иногда называется выделенными курсорами.

Объявляемые и размещаемые курсоры могут иметь динамические параметры.

Открываются и закрываются динамические курсоры, как и статически создаваемые, операторами OPEN и CLOSE. Но при открытии курсора, имеющего динамические параметры, должна быть указана фраза USING.